Part One

```
# import necessary  libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
# a. Load the data in “Gene_Expression_Data.xlsx”, “Gene_Information.csv”, and “Sample_Infor
gene_exp = pd.read_excel("C:/Users/icwin/OneDrive - Indiana University/[B-573] Prog. Sci-Inf
gene_info = pd.read_csv("C:/Users/icwin/OneDrive - Indiana University/[B-573] Prog. Sci-Info
samp_info = pd.read_csv("C:/Users/icwin/OneDrive - Indiana University/[B-573] Prog. Sci-Info
```

```
# b. Change the sample names from the “Gene_Expression_Data.xlsx”, based upon the phenotype
sample_name_mapping = {
    'GSM820516': 'tumor_1',
    'GSM820517': 'normal_1',
    'GSM820518': 'tumor_2',
    'GSM820519': 'normal_2',
    'GSM820520': 'tumor_3',
    'GSM820521': 'normal_3',
    'GSM820522': 'tumor_4',
    'GSM820523': 'normal_4',
    'GSM820524': 'tumor_5',
    'GSM820525': 'normal_5',
    'GSM820526': 'tumor_6',
    'GSM820527': 'normal_6',
    'GSM820528': 'tumor_7',
    'GSM820529': 'normal_7',
    'GSM820530': 'tumor_8',
    'GSM820531': 'normal_8',
    'GSM820532': 'tumor_9',
    'GSM820533': 'normal_9'
}
gene_exp.columns = gene_exp.columns.to_series().replace(sample_name_mapping)
```

```
# c. Split the merged data from part b, into to 2 parts, based upon their labeled phenotype
probe_id = gene_exp.columns[0]
gene_exp_tumor = gene_exp.loc[:, gene_exp.columns.str.contains('tumor') | (gene_exp.columns
gene_exp_normal = gene_exp.loc[:, gene_exp.columns.str.contains('normal') | (gene_exp.columr
```

```
# d. Compute the average expression for each probe from the 2 data sets from part c
gene_exp_normal["probe_id_avg_norm"] = gene_exp_normal.iloc[:, 1:].mean(axis = 1)
gene_exp_tumor["probe_id_avg_tumor"] = gene_exp_tumor.iloc[:, 1:].mean(axis = 1)
```

```
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\1994086955.py:2: SettingWithCopyWarning
A value is trying to be set on a copy of a slice from a DataFrame.
```

```
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_
  gene_exp_normal["probe_id_avg_norm"] = gene_exp_normal.iloc[:, 1:].mean(axis = 1)
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\1994086955.py:3: SettingWithCopyWarning
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_
  gene_exp_tumor["probe_id_avg_tumor"] = gene_exp_tumor.iloc[:, 1:].mean(axis = 1)
```

```
# e. Determine the fold change for each probe between the two groups ((Tumour – Control) / C
gene_exp_normal["fold_change"] = (gene_exp_tumor["probe_id_avg_tumor"] - gene_exp_normal["pr
gene_exp_tumor["fold_change"] = (gene_exp_tumor["probe_id_avg_tumor"] - gene_exp_normal["prc
```

```
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\906340446.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_
  gene_exp_normal["fold_change"] = (gene_exp_tumor["probe_id_avg_tumor"] - gene_exp_norm
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\906340446.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_
  gene_exp_tumor["fold_change"] = (gene_exp_tumor["probe_id_avg_tumor"] - gene_exp_norma
```

```
# f. Use the data from part e and “Gene_Information.csv” to identify all genes fold change m
gene_exp_normal["Gene_Name"] = gene_info["Symbol"]
gene_exp_normal["abs_fold_change"] = abs(gene_exp_normal["fold_change"])
gene_exp_normal["significant"] = np.where(gene_exp_normal["abs_fold_change"] > 5, "Yes", "No

gene_exp_tumor["Gene_Name"] = gene_info["Symbol"]
gene_exp_tumor["abs_fold_change"] = abs(gene_exp_tumor["fold_change"])
gene_exp_tumor["significant"] = np.where(gene_exp_tumor["abs_fold_change"] > 5, "Yes", "No")
```

```
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\2624655685.py:2: SettingWithCopyWarning
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_
  gene_exp_normal["Gene_Name"] = gene_info["Symbol"]
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\2624655685.py:3: SettingWithCopyWarning
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_
  gene_exp_normal["abs_fold_change"] = abs(gene_exp_normal["fold_change"])
```

```
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\2624655685.py:4: SettingWithCopyWarning
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_
  gene_exp_normal["significant"] = np.where(gene_exp_normal["abs_fold_change"] > 5, "Yes
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\2624655685.py:6: SettingWithCopyWarning
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_
  gene_exp_tumor["Gene_Name"] = gene_info["Symbol"]
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\2624655685.py:7: SettingWithCopyWarning
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_
  gene_exp_tumor["abs_fold_change"] = abs(gene_exp_tumor["fold_change"])
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\2624655685.py:8: SettingWithCopyWarning
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_
  gene_exp_tumor["significant"] = np.where(gene_exp_tumor["abs_fold_change"] > 5, "Yes",
```

```
gene_exp
gene_exp["probe_id_avg_tumor"] = gene_exp_tumor["probe_id_avg_tumor"]
gene_exp["fold_change"] = gene_exp_tumor["fold_change"]
gene_exp["Gene_Name"] = gene_exp_tumor["Gene_Name"]
gene_exp["abs_fold_change"] = gene_exp_tumor["abs_fold_change"]
gene_exp["significant"] = gene_exp_tumor["significant"]
gene_exp["higher_expressed"] = np.where(gene_exp["fold_change"] > 0, "tumor", "normal")


# g. Add a column to the result of part f to include if the gene was higher expressed in “No
# create sub table with all needed info
gene_exp['abs_fold_change'] = gene_exp['fold_change'].abs()
sub_gene_exp = gene_exp[gene_exp["abs_fold_change"] > 5]
sub_gene_exp["probe_id_avg_tumor"] = gene_exp_tumor["probe_id_avg_tumor"]
sub_gene_exp["fold_change"] = gene_exp_tumor["fold_change"]
sub_gene_exp["Gene_Name"] = gene_exp_tumor["Gene_Name"]
sub_gene_exp["abs_fold_change"] = gene_exp_tumor["abs_fold_change"]
sub_gene_exp["significant"] = gene_exp_tumor["significant"]
sub_gene_exp["higher_expressed"] = np.where(sub_gene_exp["fold_change"] > 0, "tumor", "norma

sub_gene_exp
```

```
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\137203892.py:5: SettingWithCopyWarni
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  sub_gene_exp["probe_id_avg_tumor"] = gene_exp_tumor["probe_id_avg_tumor"]
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\137203892.py:6: SettingWithCopyWarni
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  sub_gene_exp["fold_change"] = gene_exp_tumor["fold_change"]
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\137203892.py:7: SettingWithCopyWarni
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  sub_gene_exp["Gene_Name"] = gene_exp_tumor["Gene_Name"]
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\137203892.py:8: SettingWithCopyWarni
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  sub_gene_exp["abs_fold_change"] = gene_exp_tumor["abs_fold_change"]
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\137203892.py:9: SettingWithCopyWarni
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  sub_gene_exp["significant"] = gene_exp_tumor["significant"]
C:\Users\icwin\AppData\Local\Temp\ipykernel_1256\137203892.py:10: SettingWithCopyWarr
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
  sub_gene_exp["higher_expressed"] = np.where(sub_gene_exp["fold_change"] > 0, "tumor
```

	Probe_ID	tumor_1	normal_1	tumor_2	normal_2	tumor_3	normal_
46	ILMN_1651354	263.7130	2152.1880	27479.8100	441.8127	286.4274	741.82
1077	ILMN_1655915	877.0667	493.2354	8936.1080	695.5372	3809.3260	381.95
3481	ILMN_1666733	2907.1870	1103.2410	8392.2110	402.4640	10110.8700	317.90
3949	ILMN_1669046	23593.6100	326.3106	3465.2370	372.4652	7037.6630	259.17
6505	ILMN_1681260	306.8056	227.2110	6689.5230	214.3888	452.2915	237.01
6537	ILMN_1681462	1703.5730	225.3520	594.6810	190.9300	354.7823	232.97
7309	ILMN_1685403	19607.6400	217.0357	2133.3070	367.5093	1040.9450	263.97
7656	ILMN_1687319	405.0292	179.3568	2021.1510	181.2526	216.8601	200.32
8620	ILMN_1692223	70519.1100	4010.0440	40825.2300	4537.0180	50168.6000	9827.68
9311	ILMN_1695924	3569.4240	283.2978	3853.6370	427.7145	300.9037	315.47

9394	ILMN_1696295	2664.5230	853.3334	2327.2120	548.1557	12593.0200	378.68
10854	ILMN_1704294	2850.7540	243.7177	3302.8610	469.7225	2989.4570	240.44
11122	ILMN_1705814	1316.7350	184.5007	3075.1290	194.0636	1003.8070	273.31
13784	ILMN_1721354	1491.6760	222.0076	288.2184	192.0710	319.6802	173.15
14348	ILMN_1724686	6228.0080	558.9666	9716.9840	393.0929	7657.7240	628.65
14410	ILMN_1725090	1079.0220	824.6675	8858.9420	632.2052	2693.2000	528.63
14417	ILMN_1725139	666.7239	224.6087	2224.3340	253.5424	2854.5020	298.83
16652	ILMN_1739001	5244.9970	382.4315	18551.3900	393.1244	2156.5240	383.53
18142	ILMN_1748303	4641.6640	276.9154	351.8893	263.4604	352.1072	362.34
18962	ILMN_1753449	2094.3250	213.6866	935.9969	252.9055	1266.0130	217.36
19024	ILMN_1753830	1810.3450	218.8002	2827.5390	194.3795	2290.4830	338.47
19042	ILMN_1753954	2105.9720	242.2113	211.2522	227.8402	365.9522	339.09
19825	ILMN_1759089	21683.3500	188.4666	202.0734	288.1023	509.2545	189.09
21301	ILMN_1768469	30656.0900	211.7346	1175.3750	203.4603	434.4979	179.63
21628	ILMN_1770424	2577.4920	218.1218	286.7434	408.1161	259.6039	207.13
23206	ILMN_1780255	554.2372	194.1584	1571.4810	202.0681	315.0860	196.17
23840	ILMN_1784459	446.9836	398.5972	1173.8040	275.1535	4599.8640	299.75
24597	ILMN_1789507	419.2043	278.0077	4363.4770	218.3411	703.2089	220.02
24947	ILMN_1791711	2059.7500	214.6568	344.8824	248.9816	1468.9700	259.77
25261	ILMN_1793888	6814.3440	338.6233	6521.8480	461.7345	697.9776	1033.26
25465	ILMN_1795190	5078.0260	219.3684	3028.7730	263.7505	269.8332	248.78
26614	ILMN_1802441	1500.0060	213.4316	1022.4220	215.4766	1404.6530	202.45
26845	ILMN_1803862	692.5106	428.4529	44440.7000	425.3797	223.2777	818.01
27853	ILMN_1810172	245.0338	822.7598	1805.9990	243.9938	313.6623	437.67
28428	ILMN_1813704	16230.9500	307.8402	9973.3540	316.0457	7920.1150	316.77
35706	ILMN_1874575	3524.2850	235.1787	523.4674	266.2148	2287.5560	227.58
36711	ILMN_1881909	1267.7400	695.7442	17969.8900	1354.0440	39628.9500	1974.56
43838	ILMN_2169383	21627.5900	464.5416	12152.5300	485.7698	751.9752	1755.43
44118	ILMN_2184373	4947.9080	2711.8190	17135.3500	594.2034	19878.0500	303.04
44270	ILMN_2192072	25845.3500	196.4536	2781.4430	288.5048	1500.0060	178.89
44675	ILMN_2212878	331.5819	229.4336	2041.7550	238.6225	4807.3000	240.61

46559	ILMN_2331231	6261.3030	476.0888	1246.6720	1832.9190	6440.9520	1236.809
46560	ILMN_2331232	2383.9770	275.2569	480.5844	556.9512	2151.2780	438.823
47020	ILMN_2353161	7429.2210	366.9904	17474.7600	1679.7150	425.2169	783.044
47796	ILMN_2382679	1457.1560	242.9367	240.0609	218.8343	4250.5650	225.739

45 rows × 25 columns

Part 2

```
# a. Perform exploratory data analysis on the genes from part 1g
gene_exp_normal.head()
gene_exp_normal.info()
gene_exp_normal.describe()

gene_exp_tumor.head()
gene_exp_tumor.info()
gene_exp_tumor.describe()
```

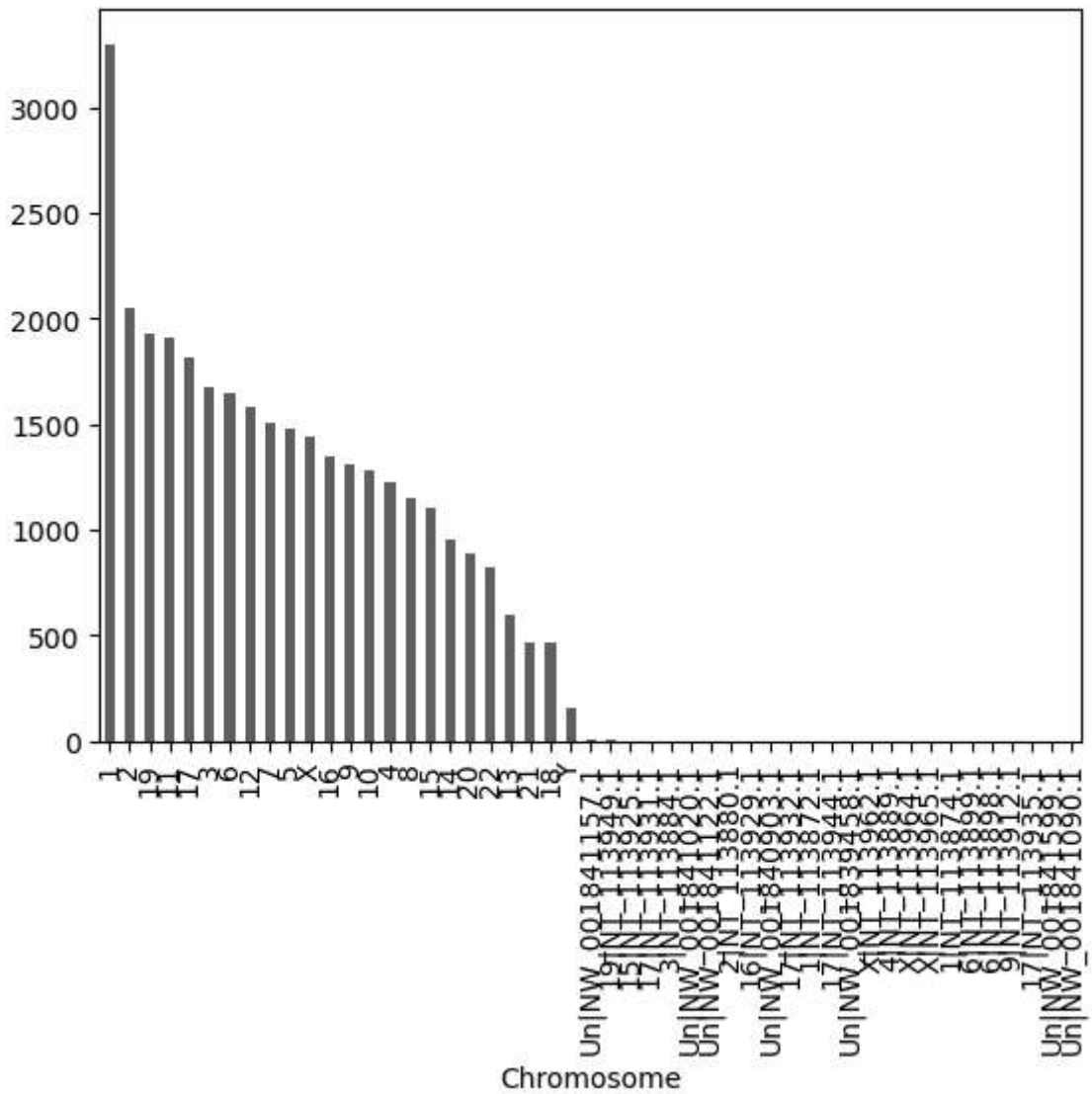
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48803 entries, 0 to 48802
Data columns (total 15 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Probe_ID           48803 non-null  object
 1   normal_1           48803 non-null  float64
 2   normal_2           48803 non-null  float64
 3   normal_3           48803 non-null  float64
 4   normal_4           48803 non-null  float64
 5   normal_5           48803 non-null  float64
 6   normal_6           48803 non-null  float64
 7   normal_7           48803 non-null  float64
 8   normal_8           48803 non-null  float64
 9   normal_9           48803 non-null  float64
 10  probe_id_avg_norm  48803 non-null  float64
 11  fold_change        48803 non-null  float64
 12  Gene_Name          36157 non-null  object
 13  abs_fold_change    48803 non-null  float64
 14  significant        48803 non-null  object
dtypes: float64(12), object(3)
memory usage: 5.6+ MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48803 entries, 0 to 48802
Data columns (total 15 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Probe_ID            48803 non-null  object
 1   tumor_1             48803 non-null  float64
 2   tumor_2             48803 non-null  float64
 3   tumor_3             48803 non-null  float64
 4   tumor_4             48803 non-null  float64
 5   tumor_5             48803 non-null  float64
 6   tumor_6             48803 non-null  float64
 7   tumor_7             48803 non-null  float64
 8   tumor_8             48803 non-null  float64
 9   tumor_9             48803 non-null  float64
 10  probe_id_avg_tumor  48803 non-null  float64
 11  fold_change         48803 non-null  float64
 12  Gene_Name           36157 non-null  object
 13  abs_fold_change     48803 non-null  float64
 14  significant         48803 non-null  object
dtypes: float64(12), object(3)
memory usage: 5.6+ MB
```

	tumor_1	tumor_2	tumor_3	tumor_4	tumor_5	tumor
count	48803.000000	48803.000000	48803.000000	48803.000000	48803.000000	48803.0000
mean	1247.095952	1247.054939	1247.081206	1247.112999	1247.149325	1247.0863
std	4647.317905	4647.325629	4647.321376	4647.314552	4647.306591	4647.3203
min	137.418400	132.615000	132.615000	132.615000	132.615000	132.6150
25%	212.073250	212.042200	212.007600	212.080200	212.120150	211.9859
50%	244.877500	244.864200	244.931500	244.939400	244.959900	244.9315

75%	508.239200	508.034300	508.239200	508.239200	508.239200	508.2993
max	71775.300000	71775.300000	71775.300000	71775.300000	71775.300000	71775.3000

```
# b. Create a histogram showing the distribution of the number of differentially expressed g
probe_id = gene_exp["Probe_ID"]
get = gene_info["Probe_ID"].isin(probe_id)
genes_inf_2 = gene_info[get]
genes_inf_2["Chromosome"].value_counts().plot(kind='bar')
```

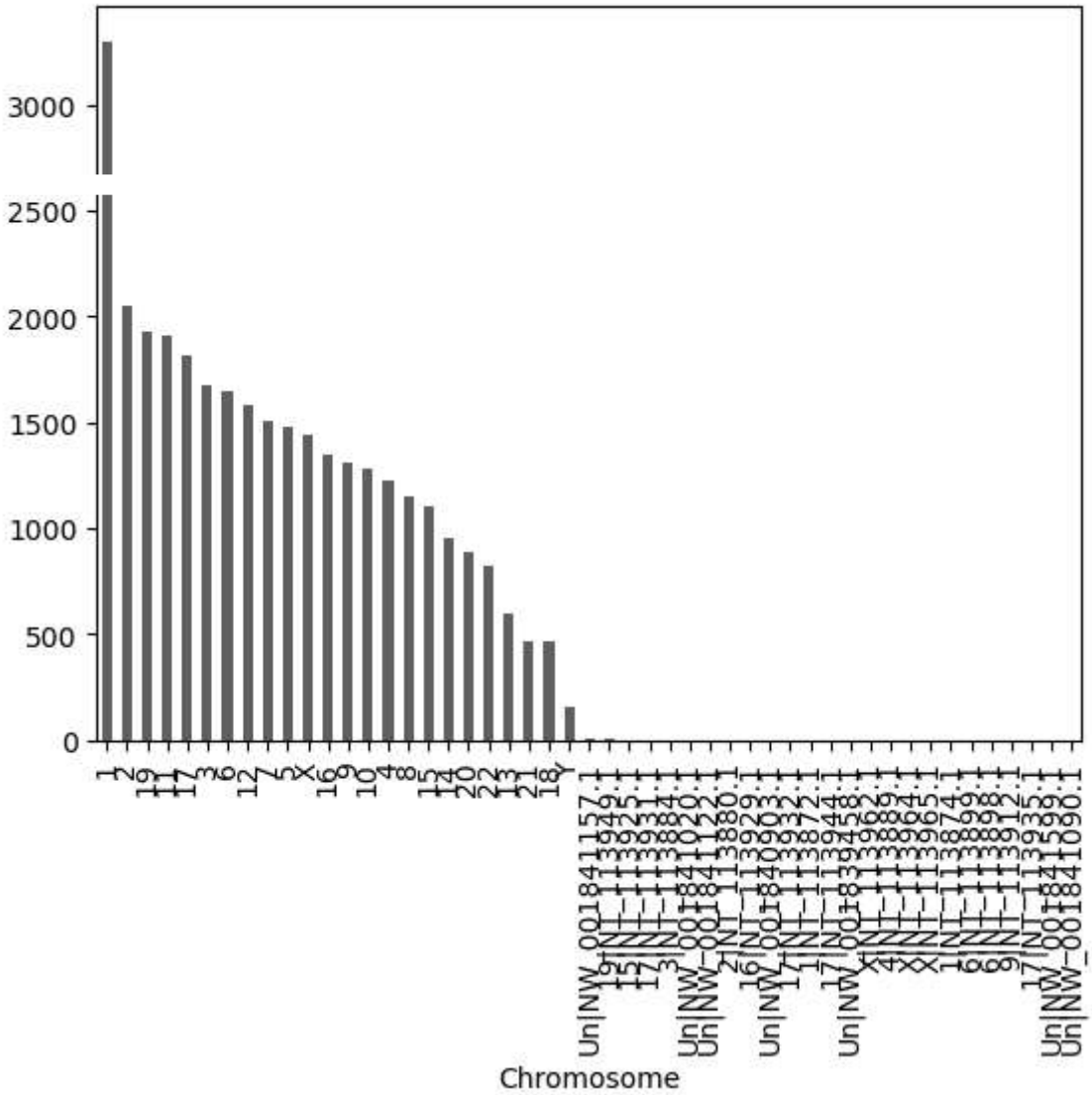
```
<Axes: xlabel='Chromosome'>
```




```
# c. Make another histogram showing the distribution of DEGs by chromosome segregated by sam
tumor_gene_exp = gene_exp[gene_exp["higher_expressed"] == "tumor"]
get = gene_info["Probe_ID"].isin(probe_id)
genes_inf_2 = gene_info[get]
genes_inf_2["Chromosome"].value_counts().plot(kind='bar')
```

```
normal_gene_exp = gene_exp[gene_exp["higher_expressed"] == "normal"]
get = gene_info["Probe_ID"].isin(probe_id)
genes_inf_2 = gene_info[get]
genes_inf_2["Chromosome"].value_counts().plot(kind='bar')
```

```
<Axes: xlabel='Chromosome'>
```




```
# d. Create a bar chart showing the percentages of the DEGs that are upregulated (higher) ir
sub_gene_exp["higher_expressed"].value_counts().plot(kind='bar')
```